

VisionBot

Webgesteuertes Fahrzeug mit Kameraübertragung

Projektdokumentation – Informationstechnik

Team:

Kevin Seeling & Eren Baser

Stand: 26.04.2026



Schaubild 1: Bild des Esp32-Cam Rc-Cars (Im Angebot auf AliExpress abgebildet)

1. Inhaltsverzeichnis

Inhaltsverzeichnis

1. Inhaltsverzeichnis.....	2
2. Zusammenfassung.....	4
3. Einleitung.....	4
3.1 Hintergrund und Motivation.....	4
3.2 Projektziel.....	5
3.3 Aufteilung der Aufgaben.....	5
3.4 Projektzeitplan (Gantt-Diagramm).....	6
4. Projektstruktur.....	8
4.1 Aufgabenstellung.....	8
4.2 Blockschaltbild – Zusammenwirken der Komponenten.....	8
4.3 Erkannte Teilprobleme.....	9
5. Hardware.....	9
5.1 Komponenten-Liste.....	9
5.2 Das 4WD-Chassis (ZYC0024).....	10
5.2.1 Aufbau und Montage des Chassis.....	10
5.3 Die TT DC-Getriebemotoren.....	11
5.3.1 Technisches Prinzip des DC-Motors.....	11
5.3.2 Motoranordnung im 4WD-System.....	11
5.3.3 Fahrtrichtungen im Detail.....	12
5.4 Der L298N H-Bridge Motortreiber.....	12
5.4.1 Warum ein Motortreiber notwendig ist.....	13
5.4.2 Das H-Bridge-Prinzip.....	13
5.4.3 Pinbelegung und Funktion des L298N-Moduls.....	14
5.4.4 Steuerlogik: HIGH/LOW an IN1-IN4.....	14
5.5 ESP32-CAM (AI-Thinker) als Hauptcontroller.....	15
5.5.1 Einschränkungen der GPIO-Pins.....	15
5.5.2 Die OV2640-Kamera.....	16
5.6 Stromversorgung: 18650-Lithium-Akkus.....	16
5.7 Die externe WLAN-Antenne.....	16
5.8 Pin-Belegung (Gesamtübersicht).....	17
5.9 Schaltplan.....	17
.....	18
5.10 Montage-Schritte.....	18
5.11 Hardware-Probleme und deren Lösungen.....	19
5.11.1 Problem: Wackelige Motoren.....	19
5.11.2 Problem: Herausfallende Motorkabel.....	19
5.11.3 Problem: Nicht befestigte Kamera.....	20
5.12 Zusammenspiel aller Hardwarekomponenten.....	20
6. Software.....	21
6.1 Software-Zweck & Gesamtarchitektur.....	21
6.1.1 Primärer Zweck des Codes.....	21
6.1.2 Architektur-Überblick.....	21
6.2 Entwicklungsumgebung & Voraussetzungen.....	21
6.3 Detaillierte Pin-Analyse & Hardware-Mapping.....	22
6.3.1 Motor-Pin-Definitionen.....	22
6.3.2 Kamera-Pin-Definitionen (AI-Thinker).....	22

Projekt: VisionBot

6.4 Setup(): Fünf Initialisierungs-Phasen.....	23
6.4.1 Phase 1: Diagnose und Sicherheitstest.....	23
6.4.2 Phase 2: Kamera-Systeminitialisierung.....	23
6.4.3 Phase 3: Kamera-Kalibrierung.....	24
6.4.4 Phase 4: WLAN-Client und Webserver.....	24
6.4.5 Phase 5: Server-Start.....	24
6.5 Loop(): Event-Architektur.....	24
6.6 Webserver-Endpunkte: Funktionsweise.....	25
6.6.1 Hauptinterface ("/").....	25
6.6.2 Kamera-Stream ("/stream").....	25
6.7 Motor-Steuerungslogik.....	26
6.8 Weboberfläche.....	26
6.8.1 Design und Layout.....	26
6.8.2 Steuerungsimplementierung.....	27
6.9 Testphase und Vorgehensweise.....	27
6.9.1 Erster Test: Motoren allein.....	27
6.9.2 Zweiter Test: Kamera-Funktion.....	27
6.9.3 Dritter Test: Zusammengeführter Gesamtcode.....	27
6.9.4 Probleme während der Testphase.....	28
6.9.5 Lösung der Probleme.....	28
6.9.6 Warum diese Testmethode sinnvoll war.....	28
7. Ergebnisse und Bewertung.....	29
7.1 Erfüllung der Anforderungen.....	29
7.2 Technische Erkenntnisse.....	29
7.3 Grenzen des Systems.....	29
8. Fazit und Ausblick.....	30
8.1 Fazit.....	30
8.2 Ausblick – Mögliche Erweiterungen.....	30
9. Nutzung der KI (ChatGpt GPT-5.3 und Claude Sonnet 4.6).....	31
9.1 Prompts und Ergebnisse (ChatGpt GPT-5.3).....	31
10. Quellenverzeichnis.....	32
11. English Summary.....	33
A. Anhang.....	34
A.1 Quellcode.....	34
B. Bedienungs-Anleitung.....	34
B.1 Voraussetzungen.....	34
B.2 Inbetriebnahme.....	34
B.3 Steuerung.....	35
B.4 Fehlerbehebung.....	35
B.5 Sicherheitshinweise.....	36

2. Zusammenfassung

Das Projekt VisionBot realisiert ein fernsteuerbares 4WD-RC-Fahrzeug auf Basis des ESP32-CAM-Mikrocontrollers. Ziel war es, ein Fahrzeug zu entwickeln, das vollständig über einen Webbrowser gesteuert werden kann und dabei in Echtzeit ein Kamerabild überträgt – ohne dass eine separate App oder eine direkte Kabelverbindung notwendig ist.

Das Fahrzeug verbindet sich automatisch mit einem WLAN-Hotspot, erhält eine IP-Adresse per DHCP und stellt einen Webserver auf Port 80 bereit. Dieser hostet eine Bedienoberfläche mit Live-Kamera-Stream (MJPEG) sowie neun Steuerrichtungen (Vorwärts, Rückwärts, Links, Rechts, vier Diagonalrichtungen sowie Stopp). Die Motorsteuerung erfolgt über eine L298N H-Bridge mit vier DC-Motoren.

Aktueller Stand: Das Fahrzeug ist als voll funktionsfähiger Prototyp fertiggestellt. Motorsteuerung, Kameraübertragung und Websteuerung funktionieren zuverlässig zusammen. Geplante Erweiterungen wie PWM-basierte Geschwindigkeitsregelung sind für einen späteren Zeitpunkt vorgesehen.

3. Einleitung

Dieses Kapitel beschreibt den Ausgangspunkt des Projekts, die konkrete Zielsetzung, die Aufgabenverteilung im Team sowie den aktuellen Entwicklungsstand.

3.1 Hintergrund und Motivation

Im Rahmen des Unterrichts im Fach Informationstechnik sollte ein praxisnahes Projekt entwickelt werden, das grundlegende Kompetenzen in den Bereichen Embedded Systems, Netzwerktechnik und Softwareentwicklung verbindet. Die Idee, ein ferngesteuertes Fahrzeug mit Kameraübertragung zu bauen, entstand aus dem Wunsch, mehrere technische Disziplinen in einem einzigen Gerät zu vereinen.

Herkömmliche RC-Fahrzeuge benötigen dedizierte Fernbedienungen mit proprietären Protokollen. Mit dem ESP32-CAM existiert jedoch eine kostengünstige, leistungsstarke Plattform, die WLAN-Kommunikation, Kamera und Mikrocontroller auf einer einzigen Platine vereint. Diese Kombination macht es möglich, ein smartes Fahrzeug zu entwickeln, das über jeden handelsüblichen Browser steuerbar ist – sei es per Smartphone, Tablet oder Computer.

Ein weiterer Motivationspunkt war die Auseinandersetzung mit Echtzeitsystemen: Wie kann ein kleiner Mikrocontroller gleichzeitig Videodaten streamen und auf Steuerbefehle reagieren? Welche Kompromisse sind dabei notwendig? Diese Fragen machen das Projekt technisch anspruchsvoll und lehrreich zugleich.

3.2 Projektziel

Das Projekt VisionBot zielt auf die Entwicklung eines vollständig webgestützten 4WD-RC-Fahrzeugs mit ESP32-CAM ab, das drei Kernkompetenzen aus dem Bereich Embedded Systems vereint: Echtzeit-Videoübertragung, WLAN-Kommunikation und Motorsteuerung.

Im Einzelnen wurden folgende Anforderungen definiert:

- Fernsteuerung über Browser – keine dedizierte App oder Fernbedienung erforderlich
- Live-Kameraübertragung mit möglichst geringer Latenz für eine präzise Navigation
- Vollständiger 4WD-Antrieb mit Differential-Lenkung (Tank-Turn) und diagonalen Fahrtrichtungen
- Automatischer WLAN-Verbindungsaufbau per DHCP – keine manuelle IP-Konfiguration notwendig
- Stabile Gleichzeitigkeit von Kamerastream und Motorsteuerung auf demselben Mikrocontroller
- Robuste und reaktionsschnelle Weboberfläche, die sowohl auf Desktop als auch auf mobilen Geräten funktioniert

Der Fokus lag dabei nicht auf einer besonders aufwendigen Mechanik, sondern auf der softwareseitigen Integration der verschiedenen Komponenten und der Lösung der dabei entstehenden technischen Herausforderungen.

3.3 Aufteilung der Aufgaben

Die Aufgaben wurden zwischen den beiden Teammitgliedern wie folgt aufgeteilt:

- Hardware (Kevin Seeling): Chassis-Montage, Motorverkabelung, Anschluss des ESP32-CAM, Verdrahtung des L298N-Motortreibers sowie Aufbau der Stromversorgung.
- Software (Eren Baser): Entwicklung des Firmware-Codes in C++ (Arduino IDE), Implementierung der Weboberfläche mit HTML/CSS/JavaScript, WLAN-Integration, Kamera-Konfiguration und Debugging.

Die enge Zusammenarbeit beider Bereiche war besonders beim Zusammenführen von Hardware und Software entscheidend: Änderungen an der Verdrahtung hatten direkten Einfluss auf den Code und umgekehrt.

Projekt: VisionBot

3.4 Projektzeitplan (Gantt-Diagramm)

Der Projektverlauf wurde in zehn Arbeitspakete (AP) gegliedert, die zeitlich aufeinander aufbauen. Das folgende Gantt-Diagramm zeigt die geplante und tatsächlich eingehaltene Zeitplanung von Anfang November 2025 bis Mitte Januar 2026.

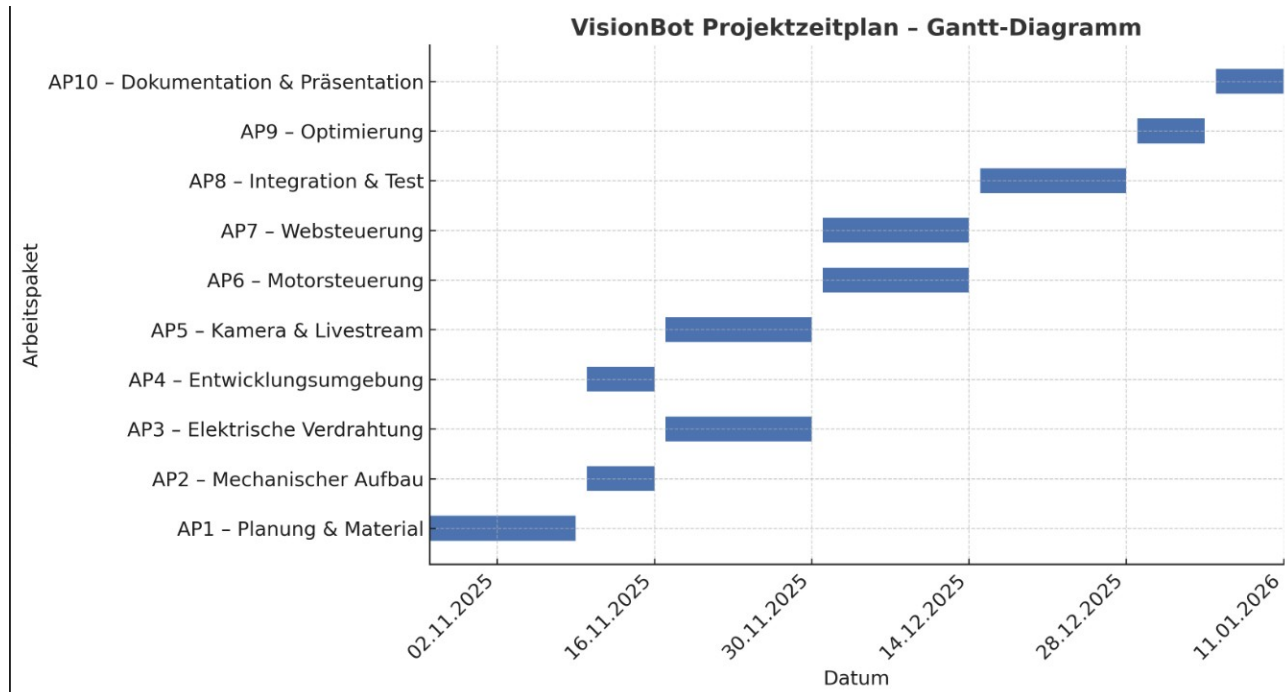


Schaubild 2: Gantt-Diagramm erstellt mit ChatGpt GPT5.3

Arbeitspaket	Bezeichnung	Zeitraum (geplant)	Beschreibung
AP1	Planung & Material	02.11. – 15.11.2025	Anforderungsanalyse, Komponentenauswahl, Bestellung des ZYC0024-Kits
AP2	Mechanischer Aufbau	16.11. – 22.11.2025	Chassis-Montage, Motoren einbauen, Räder befestigen
AP3	Elektrische Verdrahtung	16.11. – 30.11.2025	Motorkabel anschliessen, L298N verdrahten, Stromversorgung aufbauen
AP4	Entwicklungsumgebung	16.11. – 22.11.2025	Arduino IDE einrichten, ESP32-Board installieren, CH340-Treiber
AP5	Kamera & Livestream	23.11. – 02.12.2025	OV2640 initialisieren, JPEG-Stream testen, Bildausrichtung korrigieren

Projekt: VisionBot

Arbeitspaket	Bezeichnung	Zeitraum (geplant)	Beschreibung
AP6	Motorsteuerung	30.11. – 14.12.2025	H-Bridge-Logik implementieren, alle 9 Fahrtrichtungen testen
AP7	Websteuerung	30.11. – 14.12.2025	HTML/CSS/JS-Interface entwickeln, fetch()-Steuerung implementieren
AP8	Integration & Test	14.12. – 28.12.2025	Kamera + Motorsteuerung zusammenführen, Gesamtsystem testen
AP9	Optimierung	28.12.2025 – 07.01.2026	Auflösung reduzieren, Latenz verbessern, Steuerung stabilisieren
AP10	Dokumentation & Präsentation	07.01. – 16.01.2026	Projektdokumentation verfassen, Präsentation vorbereiten

Die Arbeitspakete AP2 und AP4 wurden parallel bearbeitet, da Hardware-Aufbau (Kevin) und Software-Vorbereitung (Eren) voneinander unabhängig waren. Ab AP5 nahm die Zusammenarbeit zu, da Software-Entscheidungen direkte Auswirkungen auf die Hardware-Konfiguration hatten und umgekehrt. Die längste Phase war AP8 (Integration & Test), was die technische Komplexität des Zusammenspiels aller Komponenten widerspiegelt.

Zum Zeitpunkt der Dokumentation befindet sich das Projekt in einem voll funktionsfähigen Prototypen-Zustand. Im Einzelnen sind folgende Meilensteine erreicht:

- Hardware vollständig aufgebaut und verkabelt
- Spannungsversorgung eingerichtet, alle vier Motoren funktionsfähig
- ESP32-CAM hostet Webseite mit Live-Kamerastream
- Steuerung des Fahrzeugs per Webbrowser ohne Kabelverbindung möglich
- Neun Fahrtrichtungen (inkl. Diagonalbewegungen) implementiert
- Keyboard-Steuerung über WASD-Tasten funktioniert

Noch ausstehend bzw. für spätere Versionen geplant:

- PWM-basierte Geschwindigkeitsregelung (stufenlose Drehzahlsteuerung)
- Verbesserte Weboberfläche mit Statusanzeige und Konnektivitätsmeldungen
- Optimierung der Kameralatenz durch angepasste Stream-Parameter

4. Projektstruktur

Dieses Kapitel beschreibt die Aufgabenstellung des Projekts, das Zusammenwirken aller Komponenten als Blockschaltbild sowie die Teilprobleme, die während der Entwicklung erkannt und gelöst wurden.

4.1 Aufgabenstellung

Ziel des Projekts war die Entwicklung eines ferngesteuerten 4WD-Fahrzeugs, das vollständig über einen Webbrowser bedient werden kann. Das Fahrzeug soll ohne dedizierte App oder Fernbedienung steuerbar sein und dabei in Echtzeit ein Kamerabild übertragen.

Die Kernfunktionen des VisionBot im Überblick:

- Webcam-Livestream: Der Fahrer sieht in Echtzeit, was die Kamera des Fahrzeugs aufnimmt.
- 9-Richtungs-Steuerung: Vorwärts, Rückwärts, Links, Rechts sowie vier Diagonalrichtungen über Browser-Buttons oder WASD-Tastatur.
- WLAN-Betrieb: Das Fahrzeug verbindet sich automatisch mit einem Hotspot – keine Kabelverbindung nötig.
- Plattformunabhängig: Die Steuerseite funktioniert auf Smartphone, Tablet und PC, da sie reines HTML/CSS/JS ist.

4.2 Blockschaltbild – Zusammenwirken der Komponenten

Das folgende Blockschaltbild zeigt das Zusammenwirken aller Hardware- und Software-Komponenten des VisionBot. Der Signalfluss verläuft vom Nutzer (Browser) über das WLAN zum ESP32-CAM, der sowohl die Motorsteuerung als auch den Kamerastream koordiniert.

Schicht	Komponente	Aufgabe
Benutzerinterface	Browser (HTML/CSS/JS)	Steuerbuttons + Kamerabild anzeigen
Kommunikation	WLAN (2,4 GHz)	HTTP-Requests und JPEG-Stream übertragen
Steuereinheit	ESP32-CAM (AI-Thinker)	Webserver, Kamera, GPIO-Steuerung
Motoransteuerung	L298N H-Bridge	Schwache GPIO-Signale in Motorstrom umwandeln
Antrieb	4x TT DC-Getriebemotor	Mechanische Bewegung des Fahrzeugs
Bilderfassung	OV2640 Kamera (im ESP32-CAM)	JPEG-Bilder aufnehmen und liefern

Projekt: VisionBot

Schicht	Komponente	Aufgabe
Energieversorgung	2x 18650-Akku (7,4V)	Gesamtes System mit Strom versorgen

4.3 Erkannte Teilprobleme

Zu Beginn des Projekts wurden die wesentlichen Teilprobleme identifiziert, die voneinander getrennt gelöst werden mussten, bevor eine Integration möglich war:

- Teilproblem 1 – Motorsteuerung: Wie können vier DC-Motoren über einen Mikrocontroller in beide Richtungen angesteuert werden? Lösung: L298N H-Bridge als Motortreiber.
- Teilproblem 2 – Kameraübertragung: Wie kann ein Kamerabild in Echtzeit über WLAN an einen Browser gesendet werden? Lösung: JPEG-Poll-Stream über HTTP.
- Teilproblem 3 – Parallelverarbeitung: Wie können Kamerastream und Motorsteuerung gleichzeitig auf einem Mikrocontroller laufen? Lösung: Non-blocking Webserver-Architektur (server.handleClient() ohne delay()).
- Teilproblem 4 – Pin-Knappheit: Der ESP32-CAM hat kaum freie GPIO-Pins, da die meisten von der Kamera belegt sind. Lösung: Vier Pins (2, 13, 14, 15) sorgfältig ausgewählt, die nicht mit der Kamera kollidieren.
- Teilproblem 5 – Mechanische Stabilität: Kabel und Motoren lösten sich durch Vibrationen. Lösung: Verlöten der Motoranschlüsse, festere Verschraubung.

5. Hardware

Dieses Kapitel beschreibt alle eingesetzten Hardwarekomponenten des VisionBot im Detail, erklärt wie sie zusammenwirken, dokumentiert die aufgetretenen mechanischen Probleme und deren Lösungen.

5.1 Komponenten-Liste

Komponente	Menge	Spezifikation
ESP32-CAM (AI-Thinker)	1	ESP32-S Chip, OV2640 Kamera, 4MB Flash, 4MB PSRAM
L298N H-Bridge Motortreiber	1	Dualer Vollbrücken-Treiber, 5-35V, max. 2A pro Kanal
TT DC-Getriebemotor	4	3-6V DC, TT-Standardbauform, Kunststoffgetriebe
4WD-Acryl-Chassis (ZYC0024)	1	Zwei-Ebenen-Aufbau, Acrylplatten, Metallhalterungen

Projekt: VisionBot

Komponente	Menge	Spezifikation
18650 Lithium-Akku	2	3,7V / 2200mAh, in Reihe: ca. 7,4V
18650 Batteriehalter	1	2-Zellen-Halter mit Anschlusskabeln
Externe WLAN-Antenne	1	2,4 GHz, IPX/u.FL-Anschluss, im Kit enthalten
Dupont-Kabel / Jumper Wire	mehrere	Verbindungskabel zwischen ESP32-CAM und L298N

5.2 Das 4WD-Chassis (ZYC0024)

Das Chassis des VisionBot basiert auf einem handelsüblichen Zwei-Ebenen-Acryl-Roboter-Fahrgestell (Modell ZYC0024). Es besteht aus zwei transparenten Acrylplatten, die durch Abstandshalter (Spacer) aus Metall miteinander verbunden werden. Die untere Platte nimmt die vier DC-Motoren auf, die in Metallhalterungen (Motor-Brackets) an den vier Ecken verschraubt werden. Die obere Platte dient als Montagefläche für den L298N-Motortreiber, den Batteriehalter und den ESP32-CAM.

Das Chassis besteht aus zwei transparenten Acrylplatten, die mit Metallabstandshaltern auf Abstand gehalten werden. Die Acrylplatten sind mit vorgestanzten Löchern für die Motorhalterungen, Spacer und weitere Komponenten versehen. Die genauen Abmessungen sind dem offiziellen Aufbauguide des ZYC0024-Kits zu entnehmen (Ordner: 1_Get_start/1_Assembly_guide).

5.2.1 Aufbau und Montage des Chassis

Der Zusammenbau erfolgt in mehreren Schritten:

- Schritt 1 – Motoren einbauen: Jeder der vier TT-DC-Motoren wird mit zwei Schrauben in einer Metallhalterung (L-förmiges Bracket) befestigt. Diese Halterungen werden anschließend an den vier Ecken der unteren Acrylplatte verschraubt. Dabei ist zu beachten, dass jeweils zwei Motoren spiegelbildlich montiert werden, damit alle vier Räder in dieselbe physische Richtung drehen, wenn die gleiche elektrische Polung angelegt wird.
- Schritt 2 – Räder aufsetzen: Die Kunststoffräder werden auf die D-förmige Motorwelle geschoben. Die abgeflachte Seite der D-Welle muss mit der entsprechenden Aussparung im Rad fluchten. Danach wird die Radhalterungsschraube festgezogen, um das Rad zu fixieren.
- Schritt 3 – Spacer montieren: Die Abstandshalter (typischerweise M3-Messingspacer, 30 mm Länge) werden an den Bohrungen der unteren Platte befestigt. Sie stellen den Abstand zur oberen Platte sicher.
- Schritt 4 – Obere Platte aufsetzen: Die obere Acrylplatte wird auf die Spacer aufgelegt und mit M3-Schrauben gesichert.
- Schritt 5 – Komponenten platzieren: L298N-Modul, Batteriehalter und ESP32-CAM werden auf der oberen Platte positioniert und mit Schrauben, Klettverschluss oder Kabelbindern befestigt.

5.3 Die TT DC-Getriebemotoren

Das Fahrzeug verwendet vier baugleiche TT DC-Getriebemotoren. Diese Motoren sind in der Hobby-Robotik weit verbreitet und bieten ein gutes Verhältnis aus Drehmoment, Geschwindigkeit und Preis. Der Begriff TT beschreibt die standardisierte Bauform mit dem charakteristischen gelben Kunststoffgehäuse und der D-förmigen Ausgangswelle.

5.3.1 Technisches Prinzip des DC-Motors

Ein DC-Motor (Gleichstrommotor) wandelt elektrische Energie in mechanische Rotationsenergie um. Im Inneren des Motors befinden sich zwei Hauptkomponenten: Der Stator (feststehende Permanentmagnete) und der Rotor (drehende Spule, auch Anker genannt). Wenn Gleichstrom durch die Rotowicklung fließt, entsteht ein Magnetfeld, das mit dem Feld der Permanentmagnete wechselwirkt. Diese magnetische Wechselwirkung erzeugt ein Drehmoment, das den Rotor in Bewegung versetzt.

Die Richtung des Stroms durch die Spule bestimmt die Drehrichtung des Motors. Dreht man die Polung der angelegten Spannung um (Plus und Minus tauschen), dreht sich der Motor in die entgegengesetzte Richtung. Genau dieses Prinzip nutzt die H-Bridge des L298N: Sie kann die Polarität der Spannung an den Motorausgängen elektronisch umschalten, ohne dass Kabel physisch umpgesteckt werden müssen.

Das Getriebe (Getriebestufe) reduziert die hohe Drehzahl des Elektromotors auf eine für das Fahrzeug nutzbare Geschwindigkeit und erhöht dabei gleichzeitig das Drehmoment. Die genaue Getriebeuntersetzung der im ZYC0024-Kit enthaltenen TT-Motoren ist nicht im Datenblatt spezifiziert. Bei vergleichbaren TT-Motoren liegt sie typischerweise im Bereich 1:48 bis 1:120, wobei niedrigere Untersetzungen höhere Drehzahl aber weniger Drehmoment bedeuten.

5.3.2 Motoranordnung im 4WD-System

Im VisionBot sind die vier Motoren in zwei Gruppen aufgeteilt: die rechte Motorgruppe (vorne rechts + hinten rechts) und die linke Motorgruppe (vorne links + hinten links). Alle Motoren einer Gruppe werden jeweils gleich angesteuert, da sie immer synchron laufen sollen.

Wichtig bei der Montage: Da die Motoren auf beiden Seiten des Chassis gespiegelt eingebaut sind, würden zwei identisch angesteuerte Motoren in entgegengesetzter Richtung drehen, wenn man sie an dieselbe Polung anschließt. Beim ZYC0024-Chassis wird dies dadurch kompensiert, dass die Motoren einer Seite elektrisch in derselben Polarität angeschlossen werden, die der mechanischen Spiegelung Rechnung trägt. Alternativ kann dies auch im Code durch das Tauschen von HIGH/LOW-Signalen korrigiert werden – was im VisionBot-Projekt auch gemacht wurde (siehe Kapitel 6.9.5).

5.3.3 Fahrtrichtungen im Detail

Durch die gezielte Ansteuerung der beiden Motorgruppen lassen sich folgende Fahrtrichtungen realisieren:

Fahrtrichtung	Rechte Gruppe (IN1/IN2)	Linke Gruppe (IN3/IN4)	Ergebnis
Vorwärts	Vorwärts (IN2=HIGH)	Vorwärts (IN4=HIGH)	Beide Seiten drehen gleichsinnig vorwärts
Rückwärts	Rückwärts (IN1=HIGH)	Rückwärts (IN3=HIGH)	Beide Seiten drehen gleichsinnig rückwärts
Links (Tank)	Vorwärts (IN2=HIGH)	Rückwärts (IN3=HIGH)	Rechte Seite vor, linke zurück: dreht links auf der Stelle
Rechts (Tank)	Rückwärts (IN1=HIGH)	Vorwärts (IN4=HIGH)	Linke Seite vor, rechte zurück: dreht rechts auf der Stelle
Vorwärts-Links (Kurve)	Vorwärts (IN2=HIGH)	Stop (alle LOW)	Rechte Seite treibt, linke läuft frei: sanfte Linkskurve
Vorwärts-Rechts (Kurve)	Stop (alle LOW)	Vorwärts (IN4=HIGH)	Linke Seite treibt, rechte läuft frei: sanfte Rechtskurve
Rückwärts-Links	Rückwärts (IN1=HIGH)	Stop (alle LOW)	Rechte Seite rückwärts, linke frei
Rückwärts-Rechts	Stop (alle LOW)	Rückwärts (IN3=HIGH)	Linke Seite rückwärts, rechte frei
Stopp	Stop (alle LOW)	Stop (alle LOW)	Alle Motoren stromlos, High-Impedanz

Die Tank-Lenkung (reines Links/Rechts) dreht das Fahrzeug praktisch auf der Stelle um seinen Mittelpunkt. Dies ist bei einem 4WD-Fahrzeug ohne lenkbare Vorderachse die einzige Möglichkeit, die Richtung zu ändern. Die Diagonalbewegungen (Vorwärts-Links usw.) erzeugen eine weichere Kurve, weil nur eine Motorseite aktiv ist, während die andere frei mitläuft.

5.4 Der L298N H-Bridge Motortreiber

Der L298N ist das Herzstück der Motorsteuerung. Er fungiert als elektronischer Schalter zwischen dem ESP32-CAM (der nur winzige Steuerströme liefern kann) und den DC-Motoren (die deutlich höhere Ströme benötigen).

5.4.1 Warum ein Motortreiber notwendig ist

Ein Mikrocontroller wie der ESP32 liefert an seinen GPIO-Pins lediglich 3,3 V Logikspannung und kann maximal etwa 12 mA pro Pin bereitstellen. Ein DC-Motor benötigt dagegen typischerweise 200-500 mA bei 3-6 V Betriebsspannung. Ein direkter Anschluss eines Motors an einen GPIO-Pin würde den ESP32 sofort zerstören. Der L298N übernimmt deshalb die Aufgabe, die schwachen Steuersignale des Mikrocontrollers in leistungsstarke Motorsignale umzusetzen.

Parameter	Wert (Herstellerangabe)
Treiber-Chip	L298N Doppel-H-Bridge DC Motor Treiber
Motorversorgung Vs (direkt)	+5V bis +35V
Motorversorgung Vs (mit Board-Versorgung)	+7V bis +35V
Peak-Strom Motorkanal (Io)	2A pro Kanal
Logikversorgung Vss	+5V bis +7V
Logik-Arbeitsstrombereich	0 - 36 mA
Steuersignal LOW-Pegel	-0,3V bis 1,5V
Steuersignal HIGH-Pegel	2,3V bis Vss
Enable-Signal LOW (inaktiv)	-0,3V bis 1,5V
Enable-Signal HIGH (aktiv)	2,3V bis Vss
Maximale Verlustleistung	20 W (bei T=75 Grad)

5.4.2 Das H-Bridge-Prinzip

Das Kernprinzip des L298N ist die H-Bridge (H-Brücke). Dieses Schaltungskonzept hat seinen Namen von der visuellen Ähnlichkeit mit dem Buchstaben H: Vier elektronische Schalter (im L298N sind dies bipolare Transistoren) sind so angeordnet, dass der Motor im Querstück des H hängt. Durch gezieltes Schliessen von jeweils zwei der vier Schalter kann die Stromrichtung durch den Motor umgekehrt werden.

Konkret funktioniert das so: Werden der linke obere Schalter (S1) und der rechte untere Schalter (S4) geschlossen, fließt Strom von links nach rechts durch den Motor – er dreht in Richtung A. Werden stattdessen der rechte obere Schalter (S2) und der linke untere Schalter (S3) geschlossen, fließt der Strom von rechts nach links – der Motor dreht in Richtung B. Sind alle Schalter offen, fließt kein Strom und der Motor läuft frei aus (Freilauf).

Der L298N enthält zwei vollständige H-Bridge-Schaltungen (Kanal A und Kanal B), kann also gleichzeitig zwei unabhängige DC-Motoren in beide Richtungen ansteuern. Im VisionBot werden beide Kanäle für je eine Motorseite (links/rechts) genutzt, wobei die beiden Motoren einer Seite parallel an denselben Ausgang angeschlossen sind.

Projekt: VisionBot

5.4.3 Pinbelegung und Funktion des L298N-Moduls

Pin / Anschluss	Bezeichnung	Funktion im VisionBot
IN1	Eingang Kanal A, Signal 1	GPIO 2 (ESP32): Rechte Motoren vorwärts
IN2	Eingang Kanal A, Signal 2	GPIO 14 (ESP32): Rechte Motoren rückwärts
IN3	Eingang Kanal B, Signal 1	GPIO 15 (ESP32): Linke Motoren vorwärts
IN4	Eingang Kanal B, Signal 2	GPIO 13 (ESP32): Linke Motoren rückwärts
ENA	Enable Kanal A (Jumper)	Dauerhaft aktiviert (Jumper gesteckt = Vollgas)
ENB	Enable Kanal B (Jumper)	Dauerhaft aktiviert (Jumper gesteckt = Vollgas)
OUT1 / OUT2	Motorausgang Kanal A	Rechte Motorgruppe (vorne + hinten)
OUT3 / OUT4	Motorausgang Kanal B	Linke Motorgruppe (vorne + hinten)
VIN (Motorversorgung)	Motorversorgung	Akkuspannung: ca. 7,4V von 2x 18650
GND	Masse	Gemeinsame Masse mit ESP32-CAM
5V-Ausgang	Logikversorgung	Intern: 78M05 Spannungsregler liefert 5V

Laut Herstellerdatenblatt (About this kit-EN.pdf) unterstützt der L298N-Motortreiber eine Motorversorgungsspannung (V_s) von +5V bis +35V. Wird das Board selbst über den VIN-Anschluss versorgt (wie im VisionBot), benötigt er mindestens +7V. Die Peakstrombelastbarkeit pro Kanal beträgt 2A. Die Logikversorgung (V_{ss}) liegt bei +5V bis +7V, der Logikarbeitsstrombereich bei 0-36 mA. Die maximale Verlustleistung beträgt 20 W (bei 75 Grad Celsius). Der interne 78M05-Spannungsregler erzeugt aus der Akkuspannung eine stabile 5V-Logikspannung. Der L298N hat einen internen Spannungsabfall von ca. 2V, d.h. bei 7,4V Akkuspannung erhalten die Motoren effektiv nur ca. 5,4V.

5.4.4 Steuerlogik: HIGH/LOW an IN1-IN4

Die Drehrichtung jeder Motorgruppe wird durch die Kombination der Signale an den IN-Pins bestimmt. Die folgende Logik gilt für jeden der beiden Kanäle:

Projekt: VisionBot

IN-Pin A (z.B. IN1)	IN-Pin B (z.B. IN2)	Motorverhalten
LOW (0V)	HIGH (3,3V)	Motor dreht Richtung 'Vorwärts'
HIGH (3,3V)	LOW (0V)	Motor dreht Richtung 'Rückwärts'
LOW	LOW	Motor stoppt (High-Impedanz, läuft aus)
HIGH	HIGH	Motor stoppt (Kurzschluss-Bremse, hält sofort)

Im VisionBot-Code wird für den Stopp-Befehl die High-Impedanz-Variante verwendet (alle LOW), die einen sanften Auslauf der Motoren ermöglicht, statt einer harten Bremsung.

5.5 ESP32-CAM (AI-Thinker) als Hauptcontroller

Der ESP32-CAM ist das Gehirn des VisionBot. Er vereint auf einer einzigen kleinen Platine (27 x 40,5 x 4,5 mm, Gewicht 10 g laut Herstellerdatenblatt) einen leistungsfähigen 32-Bit-Doppelkernprozessor, integriertes WLAN und Bluetooth, 32 Mbit (4 MB) SPI-Flash-Speicher, 520 KB internes SRAM plus 4 MB externes PSRAM sowie ein OV2640-Kameramodul.

Auf dem Modul befindet sich ein ESP32-S-Chip, der auf zwei Xtensa LX6-Prozessorkernen mit bis zu 240 MHz Taktfrequenz läuft. Dies ist wichtig für das VisionBot, da gleichzeitig der Webserver, die Bildaufnahme, die JPEG-Kompression und die Motorsteuerung bearbeitet werden müssen.

5.5.1 Einschränkungen der GPIO-Pins

Eine besondere Herausforderung beim ESP32-CAM ist die stark eingeschränkte Verfügbarkeit freier GPIO-Pins. Von den physisch vorhandenen Anschlüssen sind nahezu alle durch die Kameraschnittstelle (OV2640) belegt. Das 8-Bit-Parallel-Daten-Interface der Kamera allein beansprucht 8 GPIO-Pins (Y2-Y9), dazu kommen Takt-, Synchronisierungs- und Konfigurationspins. Effektiv verbleiben für externe Peripherie (also für den L298N-Anschluss) nur fünf verwendbare GPIO-Pins: GPIO 2, 12, 13, 14 und 15.

Im VisionBot wurden GPIO 2, 13, 14 und 15 für die vier IN-Pins des L298N verwendet. GPIO 12 wurde bewusst nicht genutzt, da dieser Pin während des Bootens einen bestimmten Pegel benötigt (er beeinflusst den Flash-Spannungsmodus) und bei falscher Beschaltung den Start des ESP32 verhindern kann. Wichtig: Laut Herstellerdokumentation sind GPIO 14 (CLK), GPIO 15 (CMD), GPIO 2 (Data 0), GPIO 4 (Data 1 / onboard LED), GPIO 12 (Data 2) und GPIO 13 (Data 3) intern mit dem microSD-Kartenslot verbunden. Im VisionBot-Projekt wird keine SD-Karte verwendet, daher können GPIO 2, 13, 14 und 15 für die Motorsteuerung genutzt werden. Eine gleichzeitige Nutzung von SD-Karte und Motorsteuerung wäre mit dieser Pin-Belegung jedoch nicht möglich.

5.5.2 Die OV2640-Kamera

Das im ESP32-CAM integrierte OV2640 ist ein 2-Megapixel-CMOS-Bildsensor von OmniVision. Er unterstützt verschiedene Auflösungen von QQVGA (160x120 Pixel) bis zu 1600x1200 Pixel (UXGA) sowie das direkte Ausgeben von JPEG-komprimierten Bilddaten. Die Verbindung zum ESP32-Chip erfolgt über einen 8-Bit-Parallelbus, der deutlich mehr Datendurchsatz ermöglicht als serielle Schnittstellen.

Für das VisionBot wurde die QQVGA-Auflösung (160x120 Pixel) gewählt, da sie die geringste Prozessorlast verursacht und schnelle Bildwiederholraten bei stabiler WLAN-Übertragung ermöglicht. Eine höhere Auflösung (z.B. QVGA mit 320x240) führt beim ESP32-CAM zu spürbarer Verzögerung in der Motorsteuerung, weil die Bildverarbeitung zu viel CPU-Zeit beansprucht.

5.6 Stromversorgung: 18650-Lithium-Akkus

Das VisionBot wird von zwei 18650-Lithium-Ionen-Akkus in Reihenschaltung mit Strom versorgt. Die Bezeichnung 18650 gibt die physische Dimension an: 18 mm Durchmesser und 65 mm Länge. Jeder Akku liefert eine Nennspannung von 3,7V. Die genaue Kapazität hängt vom verwendeten Akku-Modell ab; im Projekt wurden Akkus mit ca. 2200 mAh eingesetzt.

Durch die Reihenschaltung addieren sich die Spannungen: $2 \times 3,7V = 7,4V$ Gesamtspannung (bei geladenem Zustand bis zu 8,4V). Diese Spannung versorgt den L298N-Motortreiber direkt. Der L298N erzeugt daraus intern die 5V-Logikspannung für seinen Betrieb. Der ESP32-CAM wird über einen eigenen 5V-Anschluss versorgt, der ebenfalls vom L298N-Modul bereitgestellt werden kann.

Laut Herstellerdatenblatt beträgt der Stromverbrauch des ESP32-CAM allein: 180 mA bei 5V (Flash deaktiviert) bzw. 310 mA bei 5V (Flash auf Maximum). Im Tiefschlaf kann er auf unter 6 mA sinken. Der Hersteller empfiehlt eine stabile 5V/2A-Stromversorgung. Der Gesamtstromverbrauch des VisionBot (ESP32-CAM + 4 Motoren über L298N) hängt stark von der Fahrintensität ab. Exakte Messungen wurden im Rahmen dieses Projekts nicht durchgeführt.

5.7 Die externe WLAN-Antenne

Standardmässig ist auf dem AI-Thinker ESP32-CAM-Modul eine integrierte PCB-Antenne verbaut. Laut Herstellerdatenblatt (SKU: ZYZ0054) hat diese eine Verstärkung von 2 dBi und deckt das 2,4-GHz-Band (2412-2484 MHz) ab. Die Sendeleistung beträgt je nach WLAN-Standard 802.11b: 17+/-2 dBm, 802.11g: 14+/-2 dBm und 802.11n: 13+/-2 dBm. Da das VisionBot auch über größere Distanzen steuerbar sein soll, wurde eine externe WLAN-Antenne angeschlossen.

Die externe Antenne wird über den kleinen IPX/u.FL-Steckverbinder auf dem ESP32-CAM-Modul angeschlossen. Damit die externe Antenne genutzt wird, muss auf der ESP32-CAM-Platine ein 0-Ohm-Widerstand (oder eine Lötbrücke) umgesetzt werden, um das Antennensignal vom internen PCB-Antennenanschluss auf den externen IPX-Anschluss umzuleiten. Dies ist eine Hardware-Modifikation, die beim AI-Thinker-Modul standardmässig vorgesehen ist.

Projekt: VisionBot

Durch die externe Antenne verbessert sich die WLAN-Reichweite gegenüber der integrierten PCB-Antenne (2 dBi) deutlich. Im praktischen Betrieb wurde festgestellt, dass die Verbindung stabiler ist und weniger Bildausfälle beim Stream auftreten, wenn das Fahrzeug sich in etwas größerer Entfernung zum Hotspot befindet.

5.8 Pin-Belegung (Gesamtübersicht)

Funktion	ESP32-CAM GPIO	Verbunden mit
Motor IN1 (rechte Gruppe vorwärts)	GPIO 2	L298N IN1
Motor IN2 (rechte Gruppe rückwärts)	GPIO 14	L298N IN2
Motor IN3 (linke Gruppe vorwärts)	GPIO 15	L298N IN3
Motor IN4 (linke Gruppe rückwärts)	GPIO 13	L298N IN4
GND (Masse)	GND	L298N GND
5V Versorgung	5V-Pin	L298N 5V-Ausgang (78M05)
Kamera PWDN	GPIO 32	OV2640 Power-Down (intern)
Kamera XCLK	GPIO 0	OV2640 Clock (intern)
Kamera Datenbus Y2-Y9	GPIO 5,18,19,21,36,39,34,35	OV2640 Parallelbus (intern)
Kamera VSYNC/HREF/PCLK	GPIO 25,23,22	OV2640 Sync-Pins (intern)
Kamera I2C (SIOD/SIOC)	GPIO 26,27	OV2640 Konfigurationsbus (intern)
Externe WLAN-Antenne	IPX/u.FL Stecker	Externe 2,4 GHz Antenne

5.9 Schaltplan

Der Schaltplan zeigt die elektrischen Verbindungen zwischen ESP32-CAM, L298N-Motortreiber und den vier DC-Motoren. Die Stromversorgung erfolgt über zwei 18650-Lithium-Akkus in Reihe (ca. 7,4V), die den L298N direkt speisen. Der L298N stellt intern eine 5V-Versorgung für die Logik bereit.

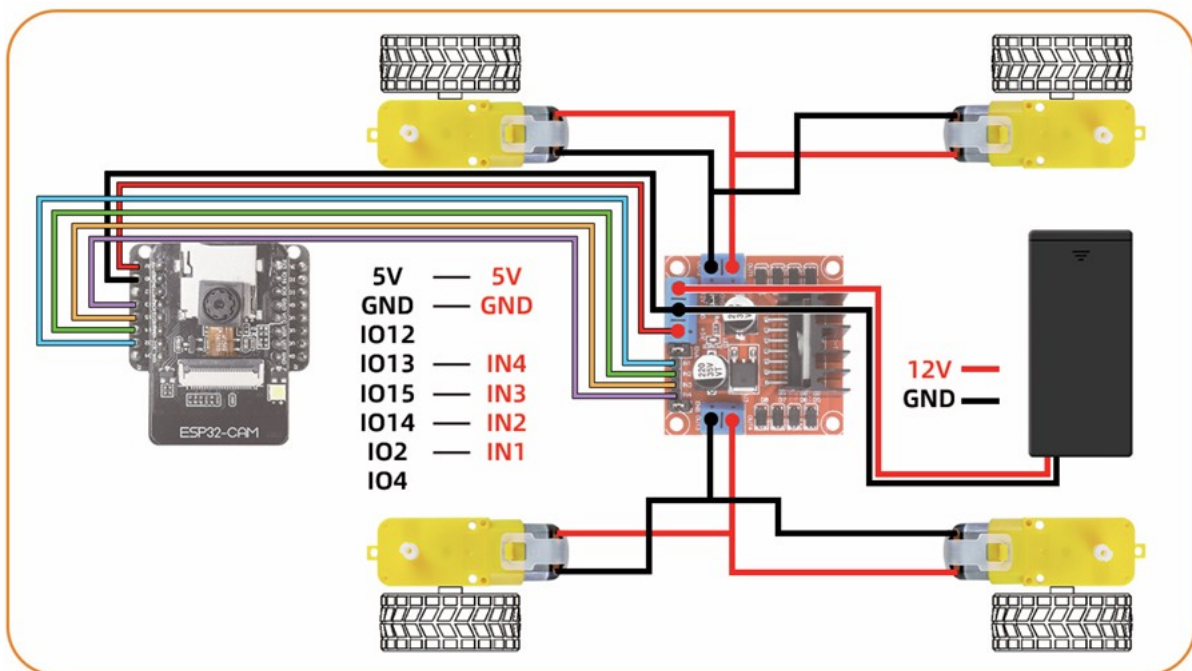


Schaubild 3: Schaltplan (In dem Herstellerdokument abgebildet unter 1_Get_start/1_Assembly_guide/24-带天线组装手册.pdf)

5.10 Montage-Schritte

Die Montage des VisionBot erfolgt in fünf Hauptphasen:

- Phase 1 – Chassis aufbauen: Untere Acrylplatte auf ebener Fläche ausrichten. Motorhalterungen (L-Brackets) an den vier Ecken mit M3-Schrauben befestigen. TT-Motoren in die Halterungen einsetzen und mit zwei Schrauben je Motor fixieren. Sicherstellen, dass die Motorwellen nach aussen zeigen.
- Phase 2 – Räder montieren: Kunststoffräder auf die D-förmigen Motorwellen aufsetzen. Die abgeflachte Seite der Welle muss in die entsprechende Nut des Rades passen. Rad mit der Sicherungsschraube (Madenschraube) fixieren, um ein Abrutschen während der Fahrt zu verhindern.
- Phase 3 – Spacer und obere Platte: Abstandshalter an den vorgesehenen Bohrungen der unteren Platte befestigen. Obere Acrylplatte aufsetzen und mit M3-Schrauben sichern.
- Phase 4 – Elektronik montieren: L298N-Modul auf der oberen Platte positionieren und befestigen. Batteriehalter montieren. ESP32-CAM auf einem Halterungsrahmen (Camera Support Frame) befestigen und auf der oberen Platte positionieren.
- Phase 5 – Verkabelung: Motorleitungen an OUT1/2 (rechte Gruppe) und OUT3/4 (linke Gruppe) des L298N anschließen. Steuerleitungen (IN1-IN4) zwischen ESP32-CAM und L298N verbinden. Akkus in den Halter einlegen und die Versorgungsleitungen anschließen. Externe Antenne am IPX-Stecker befestigen.

5.11 Hardware-Probleme und deren Lösungen

Während des Zusammenbaus und der Testphase traten mehrere mechanische und elektrische Probleme auf, die das Fahrverhalten beeinflussten.

5.11.1 Problem: Wackelige Motoren

Nach dem ersten Zusammenbau stellte sich heraus, dass die Motoren im Fahrbetrieb deutlich wackelten. Dies äusserte sich durch ungleichmässige Bewegungen, Vibrationen und instabiles Fahrverhalten, insbesondere bei schnellen Richtungswechseln. Die Ursache lag in der Art, wie die Motoren in den Halterungen befestigt waren: Die Schrauben zogen sich durch die Vibrationen des Fahrbetriebs leicht heraus, da das Acrylchassis und die Halterungen geringe Fertigungstoleranzen aufweisen.

Auswirkungen auf das Fahrverhalten: Wackelige Motoren führen dazu, dass der Abstand zwischen Motorwelle und Chassis variiert, was Schlagloch-artige Bewegungen erzeugt. Ausserdem können sich die Räder verkanten, was den Stromverbrauch erhöht und im schlimmsten Fall Motoren oder Halterungen beschädigt.

Empfohlene Lösung: Alle Motorschrauben mit erhöhtem Anzugsmoment festziehen. Sicherungsmuttern (Nyloc-Muttern) verwenden, die sich durch Vibration nicht selbst lösen. Alternativ können die Schraubengewinde mit einer kleinen Menge Gewindekleber (Loctite 243 oder vergleichbar) gesichert werden. Zusätzlich kann eine dünne Gummiunterlegscheibe zwischen Motor und Halterung die übertragene Vibration reduzieren.

5.11.2 Problem: Herausfallende Motorkabel

Die Anschlusskabel der DC-Motoren werden standardmässig einfach in die Schraubklemmen des L298N-Moduls eingesteckt. Im Betrieb mit den Motorvibrationen und bei Erschütterungen durch das Fahren kam es regelmäßig dazu, dass die Kabel aus den Klemmen rutschten. Dies führte zum kompletten Ausfall einzelner Motoren mitten im Betrieb.

Lösung – Verlöten der Verbindungen: Die Kabel wurden direkt an die Motoranschlüsse gelötet. Das Verlöten ist die technisch sauberste und zuverlässigste Lösung für diese Art von Verbindung, da eine Lötstelle mechanisch stabil ist und keinen Übergangswiderstand hat, der sich im Laufe der Zeit durch Oxidation verschlechtern kann. Beim Löten ist darauf zu achten, dass die Polung der Motorkabel korrekt ist (rotes Kabel an den positiven Ausgang OUT+, schwarzes an OUT-). Da die genaue Polung die Drehrichtung bestimmt, kann eine falsche Belegung im Code durch Tauschen der HIGH/LOW-Signale korrigiert werden.

Nach dem Verlöten wurden die Lötstellen mit Schrumpfschlauch isoliert, um Kurzschlüsse zu verhindern und die mechanische Stabilität der Verbindung weiter zu erhöhen.

5.11.3 Problem: Nicht befestigte Kamera

Die OV2640-Kamera auf der Vorderseite des ESP32-CAM-Moduls ist am AI-Thinker-Modul über einen flexiblen Flachbandkabel-Steckverbinder (FPC, Flexible Printed Circuit) mit der Hauptplatine verbunden. Dieser Steckverbinder ist durch einfaches Einstecken gesichert und kann sich bei Erschütterungen lockern. Im vorliegenden Projekt ist die Kamera vorne am Fahrzeug angebracht, jedoch nicht zusätzlich mechanisch fixiert.

Auswirkungen: Eine nicht fest befestigte Kamera kann sich während der Fahrt verdrehen, was das Kamerabild in wechselnde Richtungen kippt. Ausserdem kann bei stärkeren Erschütterungen der FPC-Steckverbinder kurzzeitig die Verbindung unterbrechen, was zu Kameraausfällen führt.

Empfohlene Lösung: Das Kameramodul sollte mit einem kleinen Kunststoffrahmen oder einem 3D-gedruckten Halter fest am Fahrzeug befestigt werden. Alternativ kann ein Tropfen Heisskleber vorsichtig an den Seiten des Kameragehäuses aufgebracht werden (nicht auf die Linse!), um es an seinem Platz zu halten. Der FPC-Steckverbinder selbst kann mit einem kleinen Stück Klebeband zusätzlich gesichert werden. Für spätere Versionen wäre ein schwenkbarer Kamerahalter (Servo-gesteuert) eine interessante Erweiterung.

5.12 Zusammenspiel aller Hardwarekomponenten

Zum Abschluss des Hardware-Kapitels ist es hilfreich, das Zusammenspiel aller Komponenten als Gesamtsystem zu verstehen:

Die Energiekette beginnt bei den 18650-Akkus, die den L298N mit ca. 7,4V versorgen. Der L298N stellt aus dieser Spannung intern 5V bereit, die auch den ESP32-CAM speisen. Der ESP32-CAM verbindet sich beim Start automatisch mit dem WLAN-Hotspot und startet seinen HTTP-Webserver. Sobald ein Nutzer im Browser die IP-Adresse des ESP32 aufruft, lädt dieser die HTML/JS-Steuerseite. Bei einem Tastendruck auf der Webseite sendet der Browser einen HTTP-GET-Request an den ESP32 (z.B. /forward). Der ESP32 empfängt diesen Request, ruft die entsprechende Motorfunktion auf und setzt die GPIO-Pins 2, 13, 14 und 15 auf die entsprechenden HIGH/LOW-Werte. Diese 3,3V-Logikpegel gehen an die IN1-IN4-Eingänge des L298N. Der L298N schaltet daraufhin seine internen H-Bridge-Transistoren und leitet den Motorstrom in die entsprechende Richtung, woraufhin die Motoren drehen und das Fahrzeug sich bewegt. Gleichzeitig nimmt der ESP32 alle ~150ms ein neues JPEG-Bild von der OV2640-Kamera auf und sendet es als HTTP-Antwort an den Browser, der es im Kamerabild-Bereich der Webseite anzeigt.

Dieses Zusammenspiel aus WLAN-Kommunikation, HTTP-Server, Kameraaufnahme und Motorsteuerung auf einem einzigen Mikrocontroller macht das Projekt technisch anspruchsvoll und zeigt die beeindruckenden Möglichkeiten moderner Embedded-Plattformen wie dem ESP32-CAM.

6. Software

6.1 Software-Zweck & Gesamtarchitektur

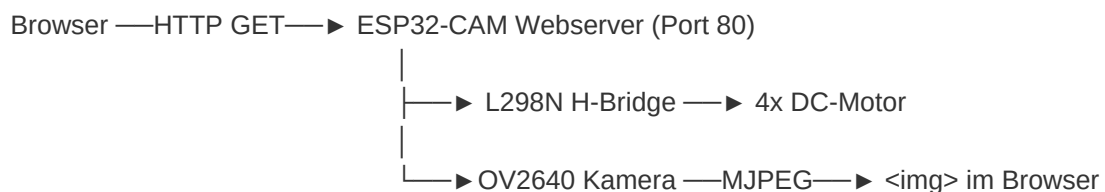
6.1.1 Primärer Zweck des Codes

Der C++-Firmware-Code für den ESP32-CAM übernimmt folgende zentralen Aufgaben:

- Hardware-Initialisierung (Motoren, Kamera, WLAN)
- Webserver-Betrieb (HTTP-Port 80 mit neun Endpunkten)
- Echtzeit-Kamera-Stream (MJPEG)
- 4WD-Motorsteuerung (neun Befehle inkl. Diagonalrichtungen)
- Non-blocking Event-Loop (parallel: Stream + Steuerung)

6.1.2 Architektur-Überblick

Die Systemarchitektur des VisionBot folgt einem einfachen Client-Server-Modell. Der ESP32-CAM agiert gleichzeitig als WLAN-Client (verbindet sich mit einem Hotspot) und als HTTP-Server für angeschlossene Browser.



Alle Funktionen laufen auf demselben Microcontroller-Kern und werden durch eine event-getriebene Loop-Architektur koordiniert. Der Webserver verwaltet eingehende HTTP-Anfragen asynchron, sodass Kamerabild und Steuerkommandos gleichzeitig verarbeitet werden können.

6.2 Entwicklungsumgebung & Voraussetzungen

Die Firmware wurde mit der Arduino IDE 2.3.8 entwickelt. Folgende Konfigurationsparameter sind für den Upload auf den ESP32-CAM notwendig:

Parameter	Einstellung
Board	AI Thinker ESP32-CAM
Partition Scheme	Huge APP (3MB No OTA/1MB SPIFFS) - laut Hersteller-Guide
CPU-Frequenz	240 MHz WiFi/BT
Flash Mode	QIO 80 MHz
PSRAM	Enabled
Upload-Geschwindigkeit	115200 Baud

Projekt: VisionBot

Verwendete Bibliotheken:

- esp_camera.h – Steuerung des OV2640-Kameramoduls
- WiFi.h – WLAN-Client-Verbindung zum Hotspot
- WebServer.h – HTTP-Server auf Port 80 mit Routing

Alle drei Bibliotheken sind Bestandteil des offiziellen ESP32-Arduino-Frameworks und müssen nicht separat installiert werden. Lediglich das ESP32-Board-Paket muss im Arduino IDE Boards Manager hinzugefügt werden.

6.3 Detaillierte Pin-Analyse & Hardware-Mapping

6.3.1 Motor-Pin-Definitionen

Die vier Motor-Steuerpins sind im Code wie folgt definiert:

```
#define IN1 2      // rechte Motorengruppe: Vorwärts
#define IN2 14     // rechte Motorengruppe: Rückwärts
#define IN3 15     // linke Motorengruppe: Vorwärts
#define IN4 13     // linke Motorengruppe: Rückwärts
```

Die Auswahl dieser Pins ist auf die eingeschränkte Pin-Verfügbarkeit des ESP32-CAM zurückzuführen. Da viele GPIO-Pins fest für die Kamera-Schnittstelle reserviert sind (Y2–Y9, VSYNC, HREF, PCLK, XCLK, SIOD, SIOC, PWDN), verbleiben nur wenige Pins für zusätzliche Peripherie. Die gewählten Pins 2, 13, 14 und 15 sind als digitale Ausgänge verwendbar und stehen nicht in Konflikt mit dem Kamerabetrieb.

6.3.2 Kamera-Pin-Definitionen (AI-Thinker)

Die Kamera-Pins sind fest durch das AI-Thinker-Boardlayout vorgegeben und müssen exakt so konfiguriert werden:

```
#define PWDN_GPIO_NUM    32    // Power Down (LOW = aktiv)
#define RESET_GPIO_NUM   -1    // Hardware-Reset (nicht genutzt)
#define XCLK_GPIO_NUM     0    // 20 MHz Clock vom ESP32
#define SIOD_GPIO_NUM     26    // I2C Data (Sensor-Konfiguration)
#define SIOC_GPIO_NUM     27    // I2C Clock (Sensor-Konfiguration)
#define Y9_GPIO_NUM       35    // MSB des Parallel-Datenbusses
#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       21
#define Y4_GPIO_NUM       19
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM        5    // LSB des Parallel-Datenbusses
#define VSYNC_GPIO_NUM    25    // Vertical Sync
#define HREF_GPIO_NUM     23    // Horizontal Reference
#define PCLK_GPIO_NUM     22    // Pixel Clock
```

Die Y2–Y9-Pins bilden den 8-Bit-Paralleldatenbus der Kamera. Über diesen Bus werden die Pixel-Daten vom OV2640-Bildsensor byteweise in den ESP32 übertragen. Die Synchronisierungs-Pins VSYNC, HREF und PCLK steuern den zeitlichen Ablauf der Bildübertragung.

6.4 Setup(): Fünf Initialisierungs-Phasen

Die setup()-Funktion wird einmalig beim Start des ESP32 ausgeführt und durchläuft fünf klar getrennte Phasen:

```
void setup();{}
```

6.4.1 Phase 1: Diagnose und Sicherheitstest

Der Serial-Monitor wird mit 115200 Baud gestartet, um Debugging-Ausgaben zu ermöglichen. Alle vier Motor-Pins werden als digitale Ausgänge konfiguriert (pinMode OUTPUT) und zunächst auf LOW gesetzt (Motorstopp).

```
Serial.begin(115200);  
pinMode(Pin, OUTPUT);  
stopMotors();
```

6.4.2 Phase 2: Kamera-Systeminitialisierung

Die Kamera wird mit folgenden Kernparametern initialisiert:

- Auflösung: QQVGA (160×120 Pixel) – optimiert für WLAN-Durchsatz und geringe Prozessorlast
- Format: JPEG mit Qualitätsstufe 20–25 (niedrige Zahl = höhere Qualität)
- PSRAM-Erkennung: Bei vorhandenem 4-MB-PSRAM werden zwei Bildpuffer aktiviert (Double-Buffering), sonst nur einer
- XCLK-Takt: 20 MHz über internen LEDC-PWM-Kanal 0

Double-Buffering verhindert, dass ein noch nicht vollständig übertragener Frame überschrieben wird. Dadurch wird das Kamerabild flüssiger und Artefakte werden reduziert. Bei Initialisierungsfehlern wird der Programmablauf mit einer Fehlermeldung im Serial-Monitor abgebrochen.

```
if (psramFound()) {  
    config.frame_size = FRAMESIZE_QQVGA;  
    config.jpeg_quality = 20;  
    config.fb_count = 2;  
} else {  
    config.frame_size = FRAMESIZE_QQVGA;  
    config.jpeg_quality = 25;  
    config.fb_count = 1; }
```

6.4.3 Phase 3: Kamera-Kalibrierung

Nach erfolgreicher Initialisierung wird das Kamerabild softwareseitig kalibriert:

- Vertikale Spiegelung (vflip = 1): Die Kamera ist physisch kopfüber montiert, daher wird das Bild um 180° gedreht.
- Horizontale Spiegelung (hmirror = 1): Korrigiert die seitenverkehrte Darstellung für eine natürliche Fahrzeugperspektive.

Diese Anpassungen werden über das `sensor_t`-Objekt vorgenommen, das direkten Zugriff auf die Register des OV2640-Bildsensors bietet.

```
sensor_t* s = esp_camera_sensor_get();  
s->set_vflip(s, 1);  
s->set_hmirror(s, 1);
```

6.4.4 Phase 4: WLAN-Client und Webserver

Das Fahrzeug verbindet sich als WLAN-Client mit dem konfigurierten Hotspot. Die Verbindung erfolgt blockierend (while-Schleife), bis eine IP-Adresse per DHCP zugewiesen wurde (typisch: ca. 5–10 Sekunden). Die zugewiesene IP wird im Serial-Monitor ausgegeben und muss im Browser eingetragen werden, um auf die Weboberfläche zuzugreifen.

```
Wifi.begin(ssid, password); while(Wifi.status() != WL_CONNECTED) { delay(500) }  
Serial.println(Wifi.localIP());
```

Anschließend werden alle HTTP-Routen registriert:

- / – Hauptseite: HTML-Seite mit eingebettetem Kamerastream und Steuerbuttons
- /stream – Kontinuierlicher JPEG-Einzel-Frame-Stream
- /forward, /back, /left, /right – Grundfahrtrichtungen
- /forward-left, /forward-right, /back-left, /back-right – Diagonalrichtungen
- /stop – Sofortiger Motorstopp

```
server.on("/", handleRoot);  
server.on("/stream", HTTP_GET, handleStream);  
server.on("/Richtung", []() { Richtung(); server.send(200, "text/plain", "OK"); });
```

6.4.5 Phase 5: Server-Start

Mit `server.begin()` wird der HTTP-Server auf Port 80 gestartet. Das System ist nun vollständig betriebsbereit und wartet auf eingehende Verbindungen. Ab diesem Zeitpunkt kann jedes Gerät im gleichen WLAN-Netz die Weboberfläche aufrufen.

```
server.begin();
```

6.5 loop(): Event-Architektur

Die `loop()`-Funktion enthält lediglich einen einzigen Aufruf: `server.handleClient()`. Diese Funktion prüft kontinuierlich, ob neue HTTP-Anfragen eingegangen sind, und ruft bei Bedarf den entsprechenden Handler auf. Da kein `delay()` verwendet wird, kann der Webserver schnell auf Anfragen reagieren – typische Reaktionszeit: unter 100 ms.

Projekt: VisionBot

Diese Architektur ermöglicht das parallele Verarbeiten von Kamerastream-Anfragen und Steuerkommandos ohne Blockaden. Der ESP32 wechselt dabei intern zwischen den verschiedenen Aufgaben, ohne dass ein echtes Betriebssystem oder RTOS notwendig ist.

```
Void loop() {  
server.handleClient(); }
```

6.6 Webserver-Endpunkte: Funktionsweise

6.6.1 Hauptinterface ("/")

Beim Aufruf der Root-URL liefert der ESP32 eine vollständige HTML-Seite. Diese enthält:

- Eine CSS-gestaltete Bedienoberfläche im responsiven Design (funktioniert auf Smartphone und Desktop)
- Ein 3×3-Button-Raster für alle neun Fahrtrichtungen (inkl. Diagonalen und Stopp)
- WASD-Tastatursteuerung: W/A/S/D für Grundrichtungen, Kombinationen (WA, WD, SA, SD) für Diagonalen
- JavaScript fetch()-Aufrufe: Steuerbefehle werden ohne Seitenneuladen als HTTP GET an den ESP32 gesendet
- Automatischer Stream-Refresh: setInterval() lädt den Kamerastream alle 150 ms neu
- Touchscreen-Unterstützung: touchstart/touchend-Events für mobile Geräte
- Safety-Stop: Beim Loslassen eines Buttons oder beim Verlassen des Browserfensters wird automatisch der Stopp-Befehl gesendet

```
void handleRoot() {  
String html = R"rawliteral(HTML-Code)rawliteral";  
server.send(200, "text/html", html); }
```

6.6.2 Kamera-Stream ("/stream")

Der Stream-Endpunkt liefert einzelne JPEG-Frames als HTTP-Antwort. Der Browser ruft diesen Endpunkt per setInterval() periodisch auf und aktualisiert das -Element mit dem jeweils aktuellen Frame. Ablauf pro Frame:

- Neues Bild vom OV2640 aufnehmen (ca. 5–10 KB als JPEG bei QQVGA)
- Frame als image/jpeg HTTP-Antwort mit Status 200 senden
- Frame-Buffer wieder freigeben (esp_camera_fb_return)

Diese Methode (Poll-basiertes JPEG statt echtem MJPEG-Multipart-Stream) wurde gewählt, weil sie besser mit der parallelen Motorsteuerung harmonisiert und weniger Ressourcen auf dem ESP32 beansprucht.

```
void handleStream() {  
camera_fb_t* fb = esp_camera_fb_get();  
if (!fb) {  
server.send(500, "text/plain", "Camera failed");  
return; }  
server.send_P(200, "image/jpeg", (const char*)fb->buf, fb->len);  
esp_camera_fb_return(fb); }
```

6.7 Motor-Steuerungslogik

Jede Fahrtfunktion setzt die vier GPIO-Pins des L298N gemäß der H-Bridge-Logik. Die Zuordnung:

Befehl	IN1 (GPIO 2)	IN2 (GPIO 14)	IN3 (GPIO 15)	IN4 (GPIO 13)
Vorwärts	LOW	HIGH	LOW	HIGH
Rückwärts	HIGH	LOW	HIGH	LOW
Links (Tank)	LOW	HIGH	HIGH	LOW
Rechts (Tank)	HIGH	LOW	LOW	HIGH
Vorwärts-Links	LOW	HIGH	LOW	LOW
Vorwärts-Rechts	LOW	LOW	LOW	HIGH
Rückwärts-Links	HIGH	LOW	LOW	LOW
Rückwärts-Rechts	LOW	LOW	HIGH	LOW
Stopp	LOW	LOW	LOW	LOW

Bei der Tank-Lenkung (Links/Rechts) drehen die beiden Motorseiten in entgegengesetzte Richtungen, wodurch das Fahrzeug auf der Stelle dreht. Bei den Diagonalbewegungen wird jeweils nur eine Motorseite aktiv angesteuert, die andere läuft frei – dies erzeugt eine weichere Kurve statt einer scharfen Tank-Drehung.

```
Void Richtung(){  
digitalWrite(pin
```

6.8 Weboberfläche

Die Weboberfläche wurde vollständig in HTML, CSS und JavaScript implementiert und wird vom ESP32-CAM direkt ausgeliefert. Es sind keine externen Server oder Frameworks notwendig.

6.8.1 Design und Layout

Die Oberfläche ist responsiv gestaltet und funktioniert sowohl auf Desktop-Browsern als auch auf mobilen Geräten. Das Layout besteht aus zwei Hauptbereichen: dem Kamerabild oben und dem Button-Raster darunter.

Das Farbschema verwendet ein warmes Beige als Hintergrund mit dunkelgrünen Akzenten für die Fahrt-Buttons und einem roten Stop-Button für die Notabschaltung. Alle Buttons sind mindestens 70px hoch, um eine gute Bedienbarkeit auf Touchscreens zu gewährleisten.

6.8.2 Steuerungsimplementierung

Die Steuerung wurde so implementiert, dass Befehle nur gesendet werden, wenn sie sich vom aktuellen Befehl unterscheiden (Deduplizierung). Dies verhindert, dass der ESP32 mit identischen Anfragen überflutet wird:

```
function sendCommand(command) {  
  if (command === activeCommand) return; // Duplikat ignorieren  
  activeCommand = command;  
  fetch('/') + command).catch(() => {}); }  
}
```

Für die Tastatursteuerung werden gedrückte Tasten in einem Set verwaltet. Bei jeder Tastenänderung (keydown/keyup) wird aus den aktuell gedrückten Tasten der entsprechende Befehl berechnet – inklusive Diagonalrichtungen bei gleichzeitig gedrückten Tasten.

6.9 Testphase und Vorgehensweise

Die Testphase des Projekts wurde bewusst in mehrere einzelne Schritte aufgeteilt. Dieser Aufbau war wichtig, damit Fehler schneller gefunden werden konnten und nicht mehrere Funktionen gleichzeitig geprüft werden mussten. Gerade bei einem Projekt, das sowohl Motorsteuerung als auch Kameraübertragung und Webserver enthält, ist es sinnvoll, zuerst einzelne Teile zu testen und erst danach alles zusammenzuführen.

6.9.1 Erster Test: Motoren allein

Am Anfang wurden zunächst nur die Motoren getestet. Dabei war das Ziel herauszufinden, ob die Verkabelung korrekt ist und ob der ESP32 die Motortreiber richtig ansteuern kann. In diesem Schritt wurde noch keine Kamera verwendet und auch kein Webserver gestartet. Der Fokus lag nur darauf, ob die Motoren vorwärts, rückwärts und beim Stop-Befehl richtig reagieren.

6.9.2 Zweiter Test: Kamera-Funktion

Nachdem die Motoren erfolgreich getestet worden waren, wurde in einem weiteren Schritt die Kamera separat geprüft. Dafür wurde ein eigener Code verwendet, der sich nur auf die Initialisierung und die Bildausgabe der Kamera konzentrierte. In dieser Phase ging es darum zu sehen, ob der ESP32-CAM das Kameramodul korrekt erkennt und ob ein stabiles Bild ausgegeben wird. Außerdem wurde geprüft, welche Auflösung sich am besten eignet und ob das Kamerabild korrekt gedreht oder gespiegelt werden muss.

6.9.3 Dritter Test: Zusammengeführter Gesamtcode

Erst nachdem Motoren und Kamera einzeln funktioniert hatten, wurden beide Teile in einem gemeinsamen Code verbunden. In diesem Schritt wurde geprüft, ob gleichzeitig ein Kamerabild übertragen und das Fahrzeug gesteuert werden kann. Das war der wichtigste Test, weil hier sichtbar wurde, ob der ESP32 die verschiedenen Aufgaben parallel bewältigen kann.

6.9.4 Probleme während der Testphase

Während des Tests traten verschiedene Schwierigkeiten auf:

- Motorrichtung: Einige Motoren drehten in die entgegengesetzte Richtung, obwohl sie korrekt programmiert waren – Ursache war eine fehlerhafte Verdrahtung der Ausgangssignale.
- Kameraausrichtung: Beim ersten Test war das Bild vertikal und horizontal gespiegelt, da die Kamera kopfüber montiert wurde.
- Systemüberlastung: Bei zu hoher Kameraauflösung kam es zu Verzögerungen im Kamerastream und in der Motorsteuerung.
- Steuerungs-Blockade: Beim ersten Zusammenführen des Codes funktionierten zwar Kamerastream und Webseite, aber die Steuerungsbuttons sendeten keine Befehle.

6.9.5 Lösung der Probleme

Lösung: Motorsteuerung

Die Steuersignale (HIGH/LOW) wurden für jede Fahrfunktion einzeln überprüft und angepasst, bis alle Motoren in der gewünschten Richtung liefen. Die Korrektur musste für jede der acht Fahrtrichtungen individuell durchgeführt werden.

Lösung: Kameraausrichtung

Durch Aktivieren von vflip und hmirror im OV2640-Sensor-Register wurde das Kamerabild korrekt ausgerichtet. Diese Einstellungen werden einmalig während der Setup-Phase gesetzt.

Lösung: Systemüberlastung

Die Kameraauflösung wurde von QVGA (320×240) auf QQVGA (160×120) reduziert. Dadurch sank die zu übertragende Datenmenge erheblich, was das System deutlich entlastete und die Latenz beider Funktionen verbesserte.

Lösung: Steuerungs-Blockade

Das Problem lag in der Ressourcenkonkurrenz zwischen Kamerastream und Webserver. Durch die Auflösungsreduzierung und den Wechsel zu einem poll-basierten Stream (statt dauerhaftem MJPEG-Multipart-Stream) wurde der Webserver entlastet. Die JavaScript-Implementierung mit fetch()-Aufrufen sorgt dafür, dass Steuerbefehle ohne Seitenneuladen gesendet werden.

6.9.6 Warum diese Testmethode sinnvoll war

Die Aufteilung in mehrere Testschritte hatte den Vorteil, dass Fehler nicht versteckt wurden. Wenn ein kompletter Code direkt auf einmal getestet wird, ist oft unklar, welcher Teil die Ursache für ein Problem ist. Durch die Trennung in Motorentest, Kamertest und Gesamttest ließ sich genau feststellen, ob die Störung von der Hardware oder von der Software kam. Außerdem konnten dadurch Änderungen gezielter vorgenommen werden, ohne andere Funktionen unnötig zu beeinflussen.

7. Ergebnisse und Bewertung

In diesem Kapitel werden die erzielten Projektergebnisse zusammengefasst und gegen die ursprünglichen Anforderungen abgeglichen.

7.1 Erfüllung der Anforderungen

Anforderung	Status	Anmerkung
Fernsteuerung per Browser	Erfüllt	9 Richtungen, WASD + Buttons
Live-Kamerastream	Erfüllt	Poll-basiertes JPEG, ~7 FPS
4WD-Antrieb mit Tank-Lenkung	Erfüllt	H-Bridge-Steuerung korrekt
Diagonale Fahrtrichtungen	Erfüllt	Alle 4 Diagonalen implementiert
Automatischer WLAN-Aufbau	Erfüllt	DHCP, IP im Serial-Monitor
Mobile Bedienbarkeit	Erfüllt	Touchscreen-Events implementiert
PWM-Geschwindigkeitsregelung	Ausstehend	Für spätere Version geplant

7.2 Technische Erkenntnisse

Das Projekt hat gezeigt, dass der ESP32-CAM trotz seiner begrenzten Ressourcen in der Lage ist, mehrere Aufgaben gleichzeitig zu bewältigen – sofern die Software entsprechend optimiert ist. Wichtige Erkenntnisse:

- Ressourcenmanagement ist entscheidend: Zu hohe Kameraauflösung beeinträchtigt die gesamte Systemperformance. Die Wahl der richtigen Bildgröße ist ein wichtiger Optimierungsparameter.
- Non-blocking Code ist unverzichtbar: delay()-Aufrufe würden den Webserver blockieren und die Steuerung unbrauchbar machen.
- Inkrementelles Testen spart Zeit: Die schrittweise Teststrategie ermöglichte es, Fehlerquellen schnell zu isolieren und zu beheben.
- JavaScript verbessert die Benutzererfahrung: fetch()-basierte Kommunikation ohne Seitenneuladen ist für Echtzeit-Steuerung essenziell.

7.3 Grenzen des Systems

Das aktuelle System hat einige bekannte Einschränkungen, die bei der Weiterentwicklung berücksichtigt werden sollten:

- WLAN-Reichweite: Die Verbindungsqualität hängt stark vom Hotspot und der Umgebung ab. Die externe WLAN-Antenne verbessert die Reichweite, kann aber bei langen Distanzen oder Hindernissen nicht alle Verbindungsprobleme lösen.
- Keine Geschwindigkeitsregelung: Alle Motoren laufen mit voller Leistung. PWM-basierte Steuerung wäre für sanftere Manöver wünschenswert.

Projekt: VisionBot

- Kameralatenz: Bei schlechter WLAN-Verbindung kann die Bildverzögerung zunehmen. Eine adaptive Auflösungssteuerung könnte hier helfen.
- Passwort im Code: Die WLAN-Zugangsdaten sind im Quellcode hardcodet. Für einen produktiven Einsatz sollte ein Konfigurationsportal (WiFiManager) implementiert werden.

8. Fazit und Ausblick

8.1 Fazit

Das Projekt VisionBot wurde erfolgreich abgeschlossen. Alle wesentlichen Projektziele wurden erreicht: Das Fahrzeug lässt sich zuverlässig über einen Webbrowser fernsteuern, überträgt in Echtzeit ein Kamerabild und reagiert schnell auf Steuerkommandos – ohne dass eine App oder eine direkte Kabelverbindung notwendig ist.

Besonders wertvoll war die Erfahrung mit den Herausforderungen eines eingebetteten Echtzeitsystems: Die begrenzte Prozessorleistung und der knappe Arbeitsspeicher des ESP32-CAM erforderten eine sorgfältige Abstimmung aller Komponenten. Die schrittweise Teststrategie und die gezielte Fehlerbehebung haben gezeigt, wie wichtig systematisches Vorgehen in der Embedded-Entwicklung ist.

Das Projekt verbindet auf anschauliche Weise mehrere Bereiche der Informationstechnik: Mikrocontroller-Programmierung, Netzkommunikation, Webentwicklung und Elektronik. Es eignet sich damit hervorragend als Lernprojekt für angehende Techniker und Informatiker.

8.2 Ausblick – Mögliche Erweiterungen

Für zukünftige Versionen des VisionBots sind folgende Erweiterungen denkbar:

- PWM-Geschwindigkeitsregelung: Durch Pulsweitenmodulation könnte die Motorgeschwindigkeit stufenlos geregelt werden. Dies würde sanftere Beschleunigungen und präzisere Manöver ermöglichen.
- WiFiManager-Integration: Statt hardcodierter WLAN-Zugangsdaten könnte ein Konfigurationsportal implementiert werden, über das Hotspot-Name und Passwort bequem per Browser eingestellt werden.
- Verbesserte Weboberfläche: Eine Statusanzeige für WLAN-Qualität, Akkustand und Temperatur würde dem Fahrer wichtige Betriebsinformationen liefern.
- Höhere Kameraauflösung: Mit einer optimierten Stream-Architektur (z.B. echtes MJPEG mit separatem Task) könnte eine höhere Auflösung bei gleichbleibender Steuerungsperformance erreicht werden.
- Hinderniserkennung: Ein Ultraschallsensor (z.B. HC-SR04) könnte Hindernisse erkennen und die Motoren automatisch stoppen, bevor es zu einer Kollision kommt.
- OTA-Updates: Over-the-Air-Firmware-Updates würden das Einspielen neuer Software ohne physischen Zugriff auf den ESP32 ermöglichen.

9. Nutzung der KI (ChatGpt GPT-5.3 und Claude Sonnet 4.6)

Zum Erstellen dieses Dokumentes wurde die Hilfe von KI (Künstliche Intelligenz) angewandt. Dabei wurde die KI ChatGpt (GPT – 5.3) verwendet um Verständnisfragen zu klären, Fehler zu beheben und den Code für die HTML-Seite so umzuschreiben, sodass es übersichtlicher und ordentlicher aussieht. Für das Erstellen von allen Texten wurde die Hilfe von Claude Sonnet 4.6 benutzt. Hierfür wurde der KI immer ein Text für den jeweiligen Abschnitt gegeben, welches dann von der KI inklusive der korrekten Rechtschreibung und Grammatik verbessert wurde. Der vorgegebene Text wurde hierbei komplett selbst geschrieben.

Der eigenständig erbrachte Anteil dieser Arbeit umfasst insbesondere die Planung und Umsetzung des Projekts. Dazu gehören der Aufbau der Hardware, die Verkabelung der einzelnen Komponenten sowie das Testen und Beheben von Fehlern während der Entwicklung. Außerdem wurde der Programmcode eigenständig angepasst, erweitert und an die Anforderungen des Projekts angepasst, insbesondere im Bereich der Motorsteuerung und der Integration des Webserver mit der Kamera. Die grundlegende Funktionsweise des Systems wurde selbst erarbeitet und verstanden, während die verwendete KI lediglich unterstützend bei Verständnisfragen, Strukturierung und sprachlicher Verbesserung eingesetzt wurde.

9.1 Prompts und Ergebnisse (ChatGpt GPT-5.3)

Die Auflösung ist verkehrt rum also um 180 grad verdreht kann man das irgendwie umdrehen im code?

- Antwort Der KI:

Du musst nach der `esp_camera_init()` Funktion ein paar Zeilen hinzufügen.

```
sensor_t * s = esp_camera_sensor_get();  
s->set_vflip(s, 1); // Bild vertikal spiegeln  
s->set_hmirror(s, 1); // Bild horizontal spiegeln
```

<https://chatgpt.com>

wie kann ich in meinem code das Livebild vergrößern oder verkleinern und wenn möglich kann ich die qualität ändern?

- Antwort der KI:

Du kannst das Livebild auf zwei Arten anpassen:

Die **Größe im Browser** änderst du im HTML-Code, indem du dem Bild eine Breite gibst (z. B. `width: 400px`). Das verändert nur die Anzeige, nicht die echte Qualität.

Die **echte Bildqualität** stellst du im Code ein:

- Mit `frame_size` änderst du die Auflösung (z. B. QVGA = klein, VGA = größer, aber langsamer)
- Mit `jpeg_quality` stellst du die Qualität ein (kleinere Zahl = bessere Qualität, aber mehr Belastung)

Zu hohe Werte können den ESP32 überlasten, deshalb ist ein Mittelwert am sinnvollsten.

<https://chatgpt.com>

Zum Erstellen des Gantt-Diagramms in 3.4 wurde auch hier Chatgpt GPT 5.3 verwendet. (Anhang der odt Datei mit einer Tabelle der Arbeitspakete und Zeitdauer)

könntest du mir zu folgender Tabelle einen Gantt-Diagramm erstellen?

-Die KI antwortete mit dem fertigen und im Abschnitt 3.4 abgebildetem Gantt-Diagramm

10. Quellenverzeichnis

Im Folgenden sind die Quellen aufgeführt, die bei der Entwicklung des Projekts genutzt wurden:

Die Anhänge sind auch auf GitHub verfügbar unter der repository:
<https://github.com/eren61ts/VisionBot.git>

Offizielle Hersteller-Dokumentation (ZYC0024-EN-With_antenna, im Anhang angehängt):

- 2-ESP32 camera Car -EN.pdf – Offizielle Anleitung des Herstellers zum ESP32 Camera Car Kit ZYC0024
- 1-DIY surveillance camera with ESP32 camera-EN.pdf – Anleitung für DIY-Kamera-Variante
- 3_datasheet/About this kit-EN.pdf – Allgemeine Kitbeschreibung und Komponentenübersicht
- 3_datasheet/ESP 32 CAM.pdf – ESP32-CAM Datenblatt des Herstellers
- 3_datasheet/ESP32-wrover_datasheet_en.pdf – ESP32-WROVER technische Referenz von Espressif
- 1_Get_start/Get started with Arduino-EN.pdf – Arduino IDE Einrichtungsanleitung des Kits
- 2_Arduino_Code/Readme.txt – Readme des mitgelieferten Originalcodes

Beispiel aus der Arduino Bibliothek Esp32 --> camera --> CameraWebServer

Weitere Quellen:

- Espressif Systems – Offizielle technische Dokumentation:
Espressif esp32-camera GitHub Repository: <https://github.com/espressif/esp32-camera>
Arduino ESP32 Bibliothek (WebServer.h): <https://github.com/espressif/arduino-esp32/tree/master/libraries/WebServer>
- OV2640 Kamerasensor:
OV2640 Camera Module Datasheet (PDF): https://www.uctronics.com/download/cam_module/OV2640DS.pdf
- Arduino IDE und Entwicklungsumgebung:
Arduino IDE Download und Dokumentation: <https://docs.arduino.cc/software/ide-v2/>
Arduino ESP32 Boardpaket – Installationsanleitung: <https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html>
Arduino Sprachreferenz (C++ für Arduino): <https://www.arduino.cc/reference/en/>

Projekt: VisionBot

- Webtechnologien (HTML, CSS, JavaScript):

MDN Web Docs – Fetch API (JavaScript):

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

MDN Web Docs – HTML Grundlagen:

https://developer.mozilla.org/en-US/docs/Learn/HTML/Introduction_to_HTML

MDN Web Docs – setInterval() für periodischen Bildabruf:

<https://developer.mozilla.org/en-US/docs/Web/API/setInterval>

- Tutorials und Lernmaterial (mit Abrufdatum 07.04.2026):

Random Nerd Tutorials – ESP32-CAM Video Streaming Web Server:

<https://randomnerdtutorials.com/esp32-cam-video-streaming-web-server-camera-home-assistant/> (abgerufen 07.04.2026)

Random Nerd Tutorials – ESP32-CAM Pinout Guide:

<https://randomnerdtutorials.com/esp32-cam-ai-thinker-pinout/> (abgerufen 07.04.2026)

Last Minute Engineers – L298N Motor Driver Tutorial:

<https://lastminuteengineers.com/l298n-dc-stepper-driver-arduino-tutorial/> (abgerufen 07.04.2026)

Dronebot Workshop – ESP32-CAM Introduction:

<https://dronebotworkshop.com/esp32-cam-intro/> (abgerufen 07.04.2026)

11. English Summary

VisionBot – English Summary Written by: Baser Eren & Kevin-Noel Seeling Date: 11th of March 2026

In this project, we built a remote-controlled RC car called VisionBot that can be fully controlled through a web browser while simultaneously streaming live footage from an integrated front-facing camera. Our goal was to combine mechanical assembly, electrical wiring, and software development into one fully functioning system. From the beginning, we divided the work according to our strengths: Kevin focused on the hardware side, including assembling the chassis, wiring the components, and mounting the camera, while Eren concentrated on the software, including the web server, camera stream, and the control interface.

On the hardware side, Kevin assembled the two-layer acrylic chassis from the ground up, attaching all four wheels to their axles and ensuring they were straight and could turn freely. He then connected the motors, power supply, and the ESP32-CAM module and mounted the camera securely at the front of the car so it faced directly forward without shaking during movement. This process was not without difficulties. At one point, several cables had been connected to the wrong pins, which caused the car to not start at all. Kevin had to disassemble the wiring almost entirely and rebuild it from scratch. Although frustrating, this experience taught him how important it is to check every detail carefully and understand how each wire affects the overall system. Once the wiring was corrected, the car came to life and all four wheels turned properly with a stable and clear camera image.

Projekt: VisionBot

On the software side, Eren set up the ESP32-CAM as both a web server and a video streamer, configured the Wi-Fi connection, defined routes for all control commands, and built the HTML interface that displays the live camera feed alongside the directional control buttons. A major challenge was combining camera streaming, motor control, and the web interface in a single program, as adding more features caused some functions to interfere with each other. Through extensive testing and debugging, the code was gradually optimized until one stable program handled all planned functions reliably.

Although the main goal has been achieved and the VisionBot works as intended, both of us see room for further improvement. Planned enhancements include smoother speed control using PWM, a more intuitive keyboard and touch-based interface, real-time indicators for connection strength and frame rate, and a more secure web server. This project gave us both valuable hands-on experience in embedded systems, networking, mechanical assembly, and problem-solving under real hardware constraints.

A. Anhang

A.1 Quellcode

Der vollständige Quellcode befindet sich in der GitHub repository (projekt_html.ino)

Die Datenblätter des Herstellers wurden ebenfalls als zip-Datei in der GitHub repository angehängt (ZYC0024-EN-With antenna.zip)

B. Bedienungs-Anleitung

B.1 Voraussetzungen

- Ein WLAN-Hotspot mit 2,4-GHz-Netz (kein 5-GHz-Netz – der ESP32 unterstützt nur 2,4 GHz)
- Ein Gerät mit Webbrowser (Smartphone, Tablet oder Computer) im gleichen WLAN
- Vollständig geladene 18650-Akkus im Fahrzeug

B.2 Inbetriebnahme

Schritt 1 – Akkus einlegen: Die beiden 18650-Zellen in den Halter einsetzen (Polung beachten) und den Schalter auf EIN stellen.

Schritt 2 – Startvorgang abwarten: Der ESP32-CAM startet automatisch. Eine Onboard-LED blinkt kurz auf. Der Startvorgang dauert ca. 10–15 Sekunden, bis die WLAN-Verbindung hergestellt ist.

Schritt 3 – IP-Adresse ermitteln: Die IP-Adresse wird im seriellen Monitor (Arduino IDE, 115200 Baud) angezeigt, sobald das WLAN verbunden ist. Alternativ kann sie im WLAN-Router nachgeschlagen werden.

Projekt: VisionBot

Schritt 4 – Browser öffnen: Auf dem Steuergerät den Browser öffnen und die IP-Adresse in die Adresszeile eingeben (z.B. <http://192.168.1.100>). Kein HTTPS notwendig.

Schritt 5 – Weboberfläche lädt: Es erscheint die VisionBot-Steuerseite mit dem Live-Kamerabild oben und den Steuerbuttons darunter.

B.3 Steuerung

Eingabe	Aktion
Button ↑ (oder Taste W)	Vorwärts fahren
Button ↓ (oder Taste S)	Rückwärts fahren
Button ← (oder Taste A)	Links drehen (Tank)
Button → (oder Taste D)	Rechts drehen (Tank)
Button ↖ (oder Tasten W+A)	Schräg vorwärts-links
Button ↗ (oder Tasten W+D)	Schräg vorwärts-rechts
Button ↙ (oder Tasten S+A)	Schräg rückwärts-links
Button ↘ (oder Tasten S+D)	Schräg rückwärts-rechts
STOP-Button (rot)	Sofortiger Motorstopp
Taste/Button loslassen	Automatischer Stopp

B.4 Fehlerbehebung

Problem	Mögliche Ursache	Lösung
Kein Kamerabild	WLAN-Verbindung unterbrochen	ESP32 neu starten (Strom trennen/verbinden)
Kamerabild verzögert	WLAN-Signal schwach	Näher an den Hotspot fahren
Auto reagiert nicht auf Buttons	Browser-Verbindung verloren	Seite neu laden (F5)
Auto fährt in falsche Richtung	Motorkabel vertauscht	Kabel am L298N-Ausgang tauschen
Webseite nicht erreichbar	Falsches WLAN-Netz (5 GHz)	Auf 2,4-GHz-Netz wechseln
ESP32 startet nicht	Akkus leer oder falsch eingelegt	Akkus prüfen und laden

B.5 Sicherheitshinweise

- Das Fahrzeug nicht auf Treppen, erhöhten Flächen oder in der Nähe von Wasser betreiben.
- Die 18650-Akkus nur mit einem geeigneten Lithium-Ionen-Ladegerät laden.
- Bei Rauch, Geruch oder starker Erwärmung sofort den Strom trennen.
- Das Fahrzeug nicht unbeaufsichtigt betreiben.

Selbstständigkeitserklärung

Hiermit erklären wir,

Kevin Seeling und **Eren Baser**

dass wir die vorliegende Projektarbeit mit dem Titel

“VisionBot”

selbstständig und nur mit den angegebenen Hilfsmitteln angefertigt haben.

Alle Stellen, die dem Sinn oder dem Wortlaut nach anderen Werken entnommen sind, wurden durch Angabe der Quellen kenntlich gemacht. Alle wörtlich entnommenen Stellen sind als Zitate gekennzeichnet.

Mannheim, den _____
Ort, Datum

Unterschrift Seeling Kevin

Mannheim, den _____
Ort, Datum

Unterschrift Baser Eren